

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 0606

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 05

Total Marks:100

Annexure A

Coding Challenge

Code of Conduct:

*I S. Dharani Teja (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

1.K-Nearest Neighbour (KNN)

Aim

To implement the K-Nearest Neighbour algorithm using Euclidean distance and classify a test instance based on user-defined K value.

CODE:

```
import numpy as np

from collections import Counter

X = [
    [1, 2],
    [2, 3],
    [3, 3],
    [6, 7],
    [7, 8],
    [8, 7]
]

y = np.array(['A', 'A', 'A', 'B', 'B', 'B'])

# Euclidean distance
def euclidean_distance(a, b):
    return np.sqrt(np.sum((a - b) ** 2))

# KNN prediction
def knn_predict(X, y, test_point, k):
    distances = []

    for i in range(len(X)):
        distance = euclidean_distance(X[i], test_point)
        distances.append((distance, y[i]))

    distances.sort()

    nearest = [label for _, label in distances[:k]]

    prediction = Counter(nearest).most_common(1)[0][0]
```

```

    return prediction

# User input
k = int(input("Enter value of K: "))

test_point = np.array([5, 6])

prediction = knn_predict(X, y, test_point, k)

print("Test Point:", test_point)
print("Predicted Class:", prediction)

# Accuracy
correct = 0

for i in range(len(X)):
    train_data = np.delete(X, i, axis=0)
    train_labels = np.delete(y, i)

    pred = knn_predict(train_data, train_labels, X[i], k)

    if pred == y[i]:
        correct += 1

accuracy = (correct / len(X)) * 100

print("Accuracy:", accuracy, "%")

```

OUTPUT:

```

Enter value of K: 4
Test Point: [5 6]
Predicted Class: B
Accuracy: 100.0 %

```

```

=== Code Execution Successful ===

```

2. Locally Weighted Regression

CODE:

```
import numpy as np

from sklearn.linear_model import LinearRegression

X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
y = np.array([2, 4, 5, 8, 10, 12])

query = 4.5

tau = 1.0

weights = np.exp(-(X.flatten() - query) ** 2) / (2 * tau ** 2))

X_matrix = np.c_[np.ones(len(X)), X.flatten()]

W = np.diag(weights)

theta = np.linalg.inv(
    X_matrix.T @ W @ X_matrix
) @ X_matrix.T @ W @ y

prediction_lwr = theta[0] + theta[1] * query

model = LinearRegression()

model.fit(X, y)

prediction_linear = model.predict([[query]])[0]

print("Query Point:", query)

print("LWR Prediction:", round(prediction_lwr, 2))

print("Linear Regression Prediction:", round(prediction_linear, 2))
```

OUTPUT:

```
... Query Point: 4.5
    LWR Prediction: 8.88
    Linear Regression Prediction: 8.86
```

3. Radial Basis Function (RBF) Network

CODE:

```
import numpy as np

X = np.array([1, 2, 3, 4, 5, 6])
y = np.array([2, 4, 6, 8, 10, 12])
centers = np.array([2, 4, 6])
```

```

width = 1.0

def gaussian(x, center, width):
    return np.exp(-((x - center) ** 2) / (2 * width ** 2))

Phi = np.zeros((len(X), len(centers)))

for i in range(len(X)):
    for j in range(len(centers)):
        Phi[i, j] = gaussian(X[i], centers[j], width)

weights = np.linalg.pinv(Phi) @ y

predictions = Phi @ weights

print("Centers:", centers)
print("Width:", width)
print("\nPredicted Outputs:")

for i in range(len(X)):
    print("Input:", X[i], "Actual:", y[i],
          "Predicted:", round(predictions[i], 2))

test = 5.0

test_phi = np.array([
    gaussian(test, c, width) for c in centers
])

test_prediction = test_phi @ weights

print("\nTest Input:", test)
print("Predicted Output:", round(test_prediction, 2))

```

OUTPUT:

```

... Centers: [2 4 6]
    Width: 1.0

    Predicted Outputs:
    Input: 1 Actual: 2 Predicted: 2.06
    Input: 2 Actual: 4 Predicted: 4.11
    Input: 3 Actual: 6 Predicted: 5.76
    Input: 4 Actual: 8 Predicted: 7.94
    Input: 5 Actual: 10 Predicted: 10.35
    Input: 6 Actual: 12 Predicted: 11.8

    Test Input: 5.0
    Predicted Output: 10.35

```

4. Case-Based Learning

CODE:

```
import numpy as np

cases = np.array([
    [25, 50000],
    [30, 60000],
    [35, 70000],
    [40, 80000],
    [45, 90000]
])

solutions = np.array([
    "Low Risk",
    "Low Risk",
    "Medium Risk",
    "Medium Risk",
    "High Risk"
])

query = np.array([33, 65000])

min_values = cases.min(axis=0)
max_values = cases.max(axis=0)

normalized_cases = (cases - min_values) / (max_values - min_values)
normalized_query = (query - min_values) / (max_values - min_values)

distances = np.sqrt(
    np.sum((normalized_cases - normalized_query) ** 2, axis=1)
)

index = np.argmin(distances)

print("New Query:", query)

print("\nDistances:")

for i in range(len(cases)):
    print("Case", i + 1, "Distance:", round(distances[i], 3))

print("\nMost Similar Case:", cases[index])

print("Predicted Solution:", solutions[index])
```

OUTPUT:

```
... New Query: [ 33 65000]

Distances:
Case 1 Distance: 0.548
Case 2 Distance: 0.195
Case 3 Distance: 0.16
Case 4 Distance: 0.513
Case 5 Distance: 0.866

Most Similar Case: [ 35 70000]
Predicted Solution: Medium Risk
```

5. K-Means Clustering

CODE:

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [1, 2],
    [1, 3],
    [2, 2],
    [8, 8],
    [9, 8],
    [8, 9]
])

k = 2

centroids = np.array([
    [1, 2],
    [8, 8]
])

for iteration in range(10):
    distances = np.zeros((len(X), k))
    for i in range(len(X)):
        for j in range(k):
            distances[i, j] = np.linalg.norm(X[i] - centroids[j])
    labels = np.argmin(distances, axis=1)
    new_centroids = np.zeros_like(centroids, dtype=float)
    for j in range(k):
        points = X[labels == j]
```

```

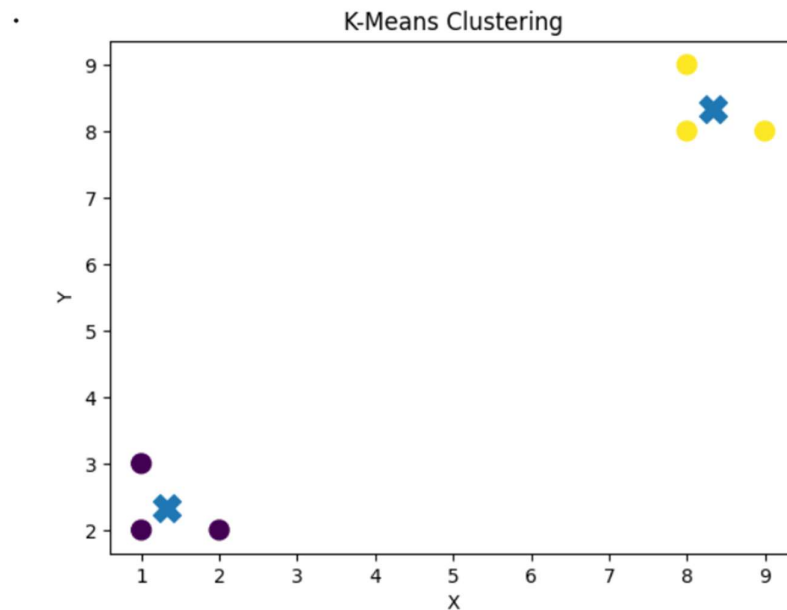
    if len(points) > 0:
        new_centroids[j] = points.mean(axis=0)
    else:
        new_centroids[j] = centroids[j]
    if np.allclose(centroids, new_centroids):
        break
    centroids = new_centroids
print("Final Centroids:")
print(centroids)
print("\nCluster Assignments:")
for i in range(len(X)):
    print(X[i], "-> Cluster", labels[i] + 1)
plt.scatter(
    X[:, 0],
    X[:, 1],
    c=labels,
    s=100
)
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker='X',
    s=200
)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("K-Means Clustering")
plt.show()

```


OUTPUT:

```
Final Centroids:
... [[1.33333333 2.33333333]
     [8.33333333 8.33333333]]
```

```
Cluster Assignments:
[1 2] -> Cluster 1
[1 3] -> Cluster 1
[2 2] -> Cluster 1
[8 8] -> Cluster 2
[9 8] -> Cluster 2
[8 9] -> Cluster 2
```



RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand